

ZTF-Orchestrator

Field Installation Guide

Version 1.2.9

Applies to: Foundation Central 1.5+ | AOS 6.5 LTS+ | Prism Central pc.2023.x

Document Title	ZTF-Orchestrator Field Installation Guide
Version	1.2.9
Date	2026-06-02
Author	John Goulden
Classification	Public

Table of Contents

Field Installation Guide.....	4
1. Architecture Overview	4
2. Prerequisites	5
2.1 Workstation.....	5
2.2 Nutanix Infrastructure.....	5
2.3 Credentials to Gather Before Starting.....	6
3. Phase 1: Foundation Central	6
3.1 Download Foundation Central	6
3.2 Deploy the Foundation Central VM.....	6
3.3 Initial Foundation Central Configuration	6
3.4 Register Foundation Central with Prism Central	7
3.5 Upload Foundation, AOS, and AHV Packages.....	7
3.6 Configure Networking for Imaging	7
3.7 Verify Foundation Central is Ready	7
4. Phase 2: ZTF-Orchestrator Installation	8
Method A: One-Command (Linux and macOS) -- Recommended	8
Method B: One-Command (Windows PowerShell).....	8
Method C: Docker	9
Method D: Linux VM Appliance on AHV	9
Method E: Manual Installation (All Platforms).....	10
5. Phase 3: First Login and Security Setup	11
5.1 Retrieve the Admin Password.....	11
5.2 First Login.....	12
5.3 Create Your Personal Account.....	12
6. Phase 4: Application Settings.....	12
6.1 Core Path Settings.....	12
6.2 Connection Profiles	12
6.3 Webhook Notifications (Optional).....	13
6.4 Verify the System Health Check.....	13
7. Phase 5: ZTF Setup and Initialisation	13
8. Phase 6: Global Configuration	14
8.1 Vault Configuration	14
8.2 IPAM Configuration	14
8.3 Foundation Central Connection	15

8.4 Prism Central Connection	15
8.5 Saving the Global Config.....	15
9. Phase 7: Workflow Config Files	15
9.1 Creating a Config File	16
9.2 Config File Reference by Workflow.....	16
10. Phase 8: Running Workflows.....	17
10.1 Single Workflow Execution	17
10.2 Re-running a Previous Execution.....	18
10.3 Multi-Script Execution.....	18
10.4 Pipeline Execution.....	18
10.5 Scheduled Execution.....	19
10.6 Parallel Multi-Site Execution	19
10.7 Approval Gates	19
10.8 Configuration Drift Detection.....	19
11. Phase 9: Production Hardening.....	19
Packer/QCOW2 validation.....	19
11.1 Automatic Startup with systemd (Linux).....	20
11.2 Automatic Startup with Docker	20
11.3 TLS Reverse Proxy with nginx.....	20
11.4 Firewall Rules (Linux ufw example).....	21
11.5 Log Rotation.....	21
12. Environment Variable Reference	22
13. Air-Gapped Deployment.....	23
13.1 Prepare Packages on an Internet-Connected Machine	23
13.2 Install on the Air-Gapped Host.....	23
13.3 Air-Gap Notes.....	23
14. Troubleshooting	24
15. Field Readiness Checklist	25
Infrastructure.....	25
ZTF-Orchestrator Installation	25
Configuration	26
Pre-Execution Validation	26
Production Readiness (team deployments).....	26
v1.2.9 Update: NKP Binary and Readiness Preparation	27

Field Installation Guide

Version: 1.2.9

Applies to: Foundation Central 1.5+, AOS 6.5 LTS+, Prism Central pc.2023.x+, ZTF-Orchestrator 1.2.9

1. Architecture Overview

ZTF-Orchestrator is a web-based GUI that sits in front of the Nutanix ZeroTouch-Framework (ZTF). It does not replace Foundation Central; it orchestrates ZTF, which calls Foundation Central for imaging workflows. Foundation Central must be deployed and reachable before any imaging workflow is executed.



Component roles at a glance:

Component	Role
ZTF-Orchestrator	Web UI and control plane: config editor, execution trigger, durable job queue, history, audit log, schedules, approvals,

	parallel execution, drift detection, and Dashboard health/status.
ZeroTouch Framework	Python CLI that translates config files into Nutanix API calls. ZTF-Orchestrator submits jobs to background workers, which invoke this CLI; Prism Central and Foundation Central connections are made by ZTF using the saved YAML configuration and credentials.
Foundation Central	Bare-metal node discovery and AOS/AHV imaging endpoint used by ZTF imaging workflows
Prism Central	Cluster-level and Prism Central operations endpoint used by ZTF configuration and PC workflows
Prism Element	Per-cluster AOS API for node-level operations

2. Prerequisites

2.1 Workstation or Jump Host

This is the machine where ZTF-Orchestrator and ZTF run. It requires network access to both Prism Central (TCP 9440) and Foundation Central (TCP 8000).

Requirement	Minimum Version	Notes
OS	Any modern Linux, macOS, or Windows	Ubuntu 22.04 LTS recommended for servers
Python	3.10	3.10, 3.11, and 3.12 are all supported
pip	any	Bundled with Python 3.10+
git	any	Required to clone both repos
Network	n/a	Must reach Prism Central on 9440 and Foundation Central on 8000
APScheduler	3.10.4	Installed through requirements.txt; enables cron schedules with polling fallback if unavailable

Docker alternative: If using the Docker installation method, only Docker Engine 24+ and docker-compose V2 are required on the host. Python, pip, and git are not needed.

2.2 Nutanix Infrastructure

Component	Minimum Version	Notes
AOS (AHV)	6.5 LTS	Earlier versions may work but are untested with ZTF
Prism Central	pc.2023.x	Required for all PC-level workflows
Foundation Central	1.5	Required for any imaging workflow
Foundation	5.3	Runs on each node; managed by Foundation Central

2.3 Credentials to Gather Before Starting

Collect these before beginning installation:

- Prism Central admin username and password
- Prism Element admin username and password (per cluster)
- Foundation Central admin username and password
- CyberArk credentials (if using CyberArk as the vault type)
- Infoblox service account credentials (if using Infoblox for IPAM)
- Internal Git mirror URL and authentication (if air-gapped)

3. Phase 1: Foundation Central

Note: If you only intend to run configuration workflows against already-deployed clusters (such as Configure Cluster, Deploy PC, or Configure PC), Foundation Central is not required, and you may proceed directly to Phase 2.

3.1 Download Foundation Central

1. Log in to the Nutanix Support Portal at portal.nutanix.com.
2. Navigate to **Downloads** and select **Foundation Central**.
3. Download the qcow2 image for AHV deployments or the **ova** for VMware ESXi deployments.

3.2 Deploy the Foundation Central VM

On AHV (recommended):

```
# Run from a CVM or using acli on the cluster
acli vm.create FoundationCentral \
  memory_mb=8192 \
  num_vcpus=4 \
  num_cores_per_vcpu=1

acli vm.disk_create FoundationCentral \
  clone_from_adfs_file=/path/to/foundation_central-<version>.qcow2

acli vm.nic_create FoundationCentral network_name=<your-vm-network>
acli vm.on FoundationCentral
```

Minimum VM sizing: 4 vCPU, 8 GB RAM, 100 GB disk.

On VMware ESXi:

Deploy the **ova** using the vSphere Client. Ensure the VM NIC is placed on a network that can reach both the bare-metal nodes to be imaged and the internet (or a local Foundation package mirror for air-gapped sites).

3.3 Initial Foundation Central Configuration

1. Identify the VM IP address from Prism Element under the VM list.
2. Open a browser and navigate to <http://<FC-VM-IP>:8000>.

3. Log in with the factory default credentials:
 - **Username:** `admin`
 - **Password:** `Nutanix/4u`
4. You will be prompted to set a new password immediately. Choose a strong password and record it in your password vault.
5. Complete the initial setup wizard.

3.4 Register Foundation Central with Prism Central

1. In Prism Central, navigate to **Settings** and select **Foundation Central**.
2. Click **Register Foundation Central**.
3. Enter the Foundation Central IP address and the admin credentials you set in step 3.3.
4. Confirm the registration. Prism Central will display Foundation Central status as **Registered** when complete.

3.5 Upload Foundation, AOS, and AHV Packages

Foundation Central requires AOS, AHV, and Foundation packages before it can image nodes.

1. In Foundation Central UI, navigate to **File Server**.
2. Upload the following packages (obtained from the Nutanix Support Portal):
 - **AOS <version>.tar** - the AOS software bundle
 - **AHV <version>.iso** - the hypervisor ISO (for AHV deployments)
 - **Foundation <version>.tar** - the Foundation package
1. Wait for each upload to complete and show a green status before proceeding.

Alternative for internet-connected sites: Foundation Central can pull packages directly from the Nutanix portal. In the File Server, click Download from Portal and authenticate with your portal credentials.

3.6 Configure Networking for Imaging

Foundation Central needs network connectivity to nodes during the imaging process.

DHCP (recommended for most sites):

Ensure the imaging VLAN has a DHCP server with a sufficient address pool. The Foundation Central VM IP must be reachable from that VLAN. Nodes receive temporary addresses during imaging; post-imaging IPs are assigned as specified in the ZTF config file.

Static / Manual:

If DHCP is not available, you will supply all node IP addresses directly in the ZTF workflow config file at execution time. No additional Foundation Central network configuration is required.

3.7 Verify Foundation Central is Ready

Before proceeding, confirm all the following:

Foundation Central UI accessible:	<code>http://<FC-VM-IP>:8000</code>
Status in Prism Central:	Registered
Packages in File Server:	AOS, AHV (if AHV deployment), Foundation all showing green
Network:	TCP 8000 reachable from the ZTF host machine

4. Phase 2: ZTF-Orchestrator Installation

ZTF-Orchestrator's installer clones and configures ZTF automatically. You do not need to install ZTF separately. Choose one of the following four methods.

Method A: One-Command (Linux and macOS) -- Recommended

```
curl -fsSL https://raw.githubusercontent.com/VirtuArchitect/ZTF-Orchestrator/main/install.sh | bash
```

With custom options:

```
ZTF_PORT=8080 ZTF_INSTALL_DIR=/opt/ztf bash install.sh
```

The script performs the following steps automatically:

1. Checks Python 3.10+, pip, and git are present
2. Clones ZTF-Orchestrator into `~/ztf/ZTF-Orchestrator`
3. Clones ZeroTouch Framework into `~/ztf/zerotouch-framework`
4. Creates a shared Python virtual environment at `~/ztf/venv`
5. Installs all Python dependencies for both components
6. Starts the server and prints the admin credentials to the terminal on first run

To restart the server after a reboot:

```
source ~/ztf/venv/bin/activate  
ZTF_PATH=~/ztf/zerotouch-framework ZTF_PORT=5001 python ~/ztf/ZTF-Orchestrator/server.py
```

Method B: One-Command (Windows PowerShell)

Open PowerShell (no Administrator rights required) and run:

```
iex ((New-Object Net.WebClient).DownloadString("https://raw.githubusercontent.com/VirtuArchitect/ZTF-Orchestrator/main/install.ps1"))
```

With a custom port:

```
$env:ZTF_PORT = "8080"  
.\install.ps1
```

Hyper-V note: Ports 4940 through 5039 may be reserved by Hyper-V on Windows. If the server fails to bind to port 5001, switch to port 8080: `$env:ZTF_PORT = "8080"` before running the installer.

To restart the server after a reboot:

```
& "$env:USERPROFILE\ztf\venv\Scripts\activate"  
python "$env:USERPROFILE\ztf\ZTF-Orchestrator\server.py"
```


Method C: Docker

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
cd ZTF-Orchestrator
docker compose up -d
docker compose logs -f # copy the admin password printed on first run
```

Windows PowerShell PostgreSQL example:

```
$env:POSTGRES_PASSWORD="change-this-postgres-password"
```

```
$env:ZTF_DATABASE_URL="postgresql://ztf:change-this-postgres-
password@postgres:5432/ztf_orchestrator"
```

```
docker compose up -d --build
```

The password in ZTF_DATABASE_URL must match POSTGRES_PASSWORD. For repeatable deployments, put the same values in .env before starting Docker Compose instead of setting them only in the current PowerShell session.

The Docker image clones ZTF during the image build step. No separate ZTF installation is required. In v1.2.9, Docker starts PostgreSQL by default and stores users, sessions, execution history, approvals, schedules, drift results, durable execution jobs, and audit events in the persistent ztf-postgres volume. Config files and operational logs remain in ztf-data and ztf-logs.

For simple file-backed testing without PostgreSQL:

```
docker compose -f docker-compose.file.yml up -d --build
```

Using a custom ZTF version or an internal Git mirror:

```
ZTF_REPO_URL=https://gitea.internal/ztf.git ZTF_REF=v2.1.0 docker compose up -d
```

Managing the Docker deployment:

Method D: Linux VM Appliance on AHV

Recommended for shared or production-like use: deploy ZTF-Orchestrator as a small Linux VM on AHV and run the appliance stack with Docker Compose. This keeps the console close to Prism Central, Foundation Central, Prism Element/CVM networks, DNS, NTP, IPAM, and internal Git repositories, while avoiding dependency on a Windows jump server session.

Recommended VM sizing: 2 vCPU, 4-8 GB RAM, 80-100 GB disk, Ubuntu Server 24.04 LTS or Rocky Linux 9, and management-network access to Nutanix control-plane services. Restrict browser access to admin or jump networks.

Quick appliance bootstrap on a fresh Linux VM:

```
sudo apt-get update
sudo apt-get install -y git curl ca-certificates
```

```
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git /opt/ztf-orchestrator-source
sudo bash /opt/ztf-orchestrator-source/appliance/scripts/firstboot.sh
```

The first-boot script installs Docker, creates `/opt/ztf-orchestrator/.env`, generates a local PostgreSQL password if one is not supplied, installs the systemd service, pulls the published container image, and starts the stack. The application admin password is still generated by ZTF-Orchestrator on first start and should be retrieved from the service logs.

The repo also contains a reference Packer template for building an AHV-importable QCOW2. Install Packer only if you intend to build and validate a distributable appliance image. Packer is best run on a Linux build VM with QEMU/KVM rather than on a Windows jump server. It is not required for normal Docker or appliance bootstrap deployment.

Large QCOW2 or OVA files should be attached to versioned GitHub Releases. Do not commit VM images or real `.env` files to the repository.

```
# Stop the container
docker compose down

# Restart after host reboot
docker compose up -d

# View logs
docker compose logs -f

# Rebuild the image (after upgrading ZTF-Orchestrator)
docker compose build --no-cache
docker compose up -d
```

Method E: Manual Installation (All Platforms)

Use this method when you need precise control over install paths or are working in a fully air-gapped environment.

Step 1: Clone both repositories

```
# Linux and macOS
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git ~/ztf/ZTF-Orchestrator
git clone https://github.com/nutanixdev/zerotouch-framework.git ~/ztf/zerotouch-framework
```

```
# Windows PowerShell
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git "$env:USERPROFILE\ztf\ZTF-Orchestrator"
git clone https://github.com/nutanixdev/zerotouch-framework.git "$env:USERPROFILE\ztf\zerotouch-framework"
```

Step 2: Create a virtual environment and install dependencies

```
# Linux and macOS
```

```
python3 -m venv ~/ztf/venv
source ~/ztf/venv/bin/activate
pip install -r ~/ztf/ZTF-Orchestrator/requirements.txt
pip install -r ~/ztf/zerotouch-framework/requirements/prod.txt
```

```
# Windows PowerShell
python -m venv "$env:USERPROFILE\ztf\venv"
& "$env:USERPROFILE\ztf\venv\Scripts\Activate.ps1"
pip install -r "$env:USERPROFILE\ztf\ZTF-Orchestrator\requirements.txt"
pip install -r "$env:USERPROFILE\ztf\zerotouch-framework\requirements\prod.txt"
```

Step 3: Set environment variables

```
# Linux and macOS: add to ~/.bashrc or ~/.zshrc, or create a .env file
export ZTF_PATH=~/.ztf/zerotouch-framework
export ZTF_PYTHON=~/.ztf/venv/bin/python3
export ZTF_PORT=5001
```

```
# Windows PowerShell: set for the session or add to your PowerShell profile
$env:ZTF_PATH = "$env:USERPROFILE\ztf\zerotouch-framework"
$env:ZTF_PYTHON = "$env:USERPROFILE\ztf\venv\Scripts\python.exe"
$env:ZTF_PORT = "5001"
```

Step 4: Start the server

```
# Linux and macOS
cd ~/ztf/ZTF-Orchestrator
python server.py
```

```
# Windows PowerShell
Set-Location "$env:USERPROFILE\ztf\ZTF-Orchestrator"
python server.py
```

Open a browser and navigate to <http://localhost:5001>. The admin password is printed to the terminal.

5. Phase 3: First Login and Security Setup

5.1 Retrieve the Admin Password

On first startup, the server generates a random admin password and prints it to the terminal:

```
=====
First run -- default admin account created
Username: admin
Password: a3f8c2e1d94b7...
Change this immediately via Settings > Users.
=====
```

Copy this password immediately. If you miss it, delete `~/.ztf-ui/users.json` (or `C:\Users\<you>\.ztf-ui\users.json` on Windows) and restart the server to generate a new one.

5.2 First Login

1. Open `http://localhost:5001` in your browser.
2. Sign in with **Username:** `admin` and the password copied from the terminal.

5.3 Create Your Personal Account

Before doing anything else, create a named admin account and retire the default:

1. Navigate to **Users** in the sidebar.
2. Click **Add User**.
3. Enter your username, a strong password, and assign the **admin** role.
4. Log out and log back in using your new account.
5. Return to **Users**, select the `admin` account, and either reset its password to something strong and unique, or delete it entirely.

5.4 Role Reference

Role	Permissions
admin	Full access: settings, global config, user management, all executions
operator	Execute workflows, manage config files, read execution history
viewer	Read-only: view configs, executions, system check

Assign roles according to the principle of least privilege. Most day-to-day users should be **operator**. Reserve **admin** for installation and user management tasks only.

6. Phase 4: Application Settings

Navigate to Settings (admin only). This page controls runtime paths, repository source, connection profiles, notifications, and product information.

6.1 Core Path Settings

Setting	Typical Linux Value	Typical Windows Value
ZTF Path	<code>~/ztf/zerotouch-framework</code>	<code>C:\Users\<you>\ztf\zerotouch-framework</code>
Python Executable	<code>~/ztf/venv/bin/python3</code>	<code>C:\Users\<you>\ztf\venv\Scripts\python.exe</code>
Config Directory	<code>~/.ztf-ui/configs</code>	<code>C:\Users\<you>\.ztf-ui\configs</code>

If you used the one-command installer or Docker, these are set automatically via environment variables. Verify them on this page regardless.

6.2 Connection Profiles

Connection Profiles centralise the endpoint and credential-reference defaults that ZTF-Orchestrator uses when generating workflow YAML. Create or update the active profile before building production workflow configs.

Recommended minimum profile fields:

- Prism Central VIP/FQDN and pc_user credential reference
- Foundation Central or PC endpoint used for imaging workflows, plus AOS/AHV image URLs
- Prism Element cluster VIP, pe_user, and cvm_credential references
- NCM endpoint and ncm_user credential reference for Calm/NCM workflows
- Directory domain/LDAP URL and service_account_credential reference where AD/LDAP configuration is used
- IPAM mode, Infoblox host/views, DNS servers, NTP servers, timezone, and site code

Use the generated ZTF defaults YAML preview on the Settings page to verify the active profile. The profile stores references and defaults only; actual passwords remain in Global Config/global.yml.

6.3 Webhook Notifications (Optional)

In Settings under the Notifications section, enter a Webhook URL to receive an HTTP POST after workflow, script, scheduled, pipeline, approval, or parallel execution events complete. The payload includes:

- Workflow or script name
- Execution status and return code
- Username that triggered the execution
- Timestamp and execution ID

This is suitable for integration with Slack, Microsoft Teams, PagerDuty, or any webhook receiver.

6.4 Verify the System Health Check

The **Dashboard** displays a system health panel immediately after login. Confirm all indicators are green:

- ZTF path exists and is a valid ZTF clone
- Python executable is reachable and has the correct version
- Config directory exists and is writable
- State Backend: confirms whether the appliance is using PostgreSQL or file-backed state and shows the database location when configured

All health indicators should be green before proceeding to workflow execution. If the state backend shows PostgreSQL, verify the database location is expected for the deployment.

In v1.2.9, the successful prerequisite check state persists while navigating during the same browser session. If the user leaves Setup & Install and returns, Step 1 remains complete until the session ends or the page state is reset.

7. Phase 5: ZTF Setup and Initialisation

Skip this phase if you used the one-command installer (Methods A or B), Docker (Method C), the Linux VM appliance (Method D), or manual installation (Method E). ZTF is already present and configured.

If you want ZTF-Orchestrator to clone and install ZTF on your behalf via the UI:

1. Navigate to **Setup and Install** in the sidebar.
2. Enter the ZTF repository URL. The default is:

<https://github.com/nutanixdev/zerotouch-framework.git>

For internal mirrors, replace this with your internal Git URL.

1. Click **Install ZTF**.
2. The terminal pane streams **git clone** and **pip install** output in real time.
3. When complete, return to the **Dashboard**. All system health indicators should now show green.

8. Phase 6: Global Configuration

Global Config is the most critical pre-flight step. It writes `<ZTF_PATH>/config/global.yml`, which ZTF reads on every workflow execution. Navigate to **Global Config** in the sidebar.

8.1 Vault Configuration

The vault setting determines how ZTF retrieves credentials (Prism passwords, FC passwords, etc.) at runtime.

Option 1: Local vault (recommended for development and small teams)

Field	Value
Vault Type	local

With the local vault, credentials are stored directly in the per-workflow YAML config files. Appropriate for isolated environments where config files are stored securely.

Option 2: CyberArk (recommended for production)

Field	Example Value
Vault Type	cyberark
CyberArk Host	cyberark.corp.example.com
Application ID	ztf-automation
Safe Name	NutanixZTF
Policy ID	ZTF-Policy

With CyberArk, workflow config files contain only the object names for password lookups, not the passwords themselves.

8.2 IPAM Configuration

Option 1: Static / Manual

Field	Value
IPAM Type	static

IP addresses are specified directly in each workflow config file. Use this option when there is no IPAM system, or for small deployments.

Option 2: Infoblox

Field	Example Value
IPAM Type	infoblox
Infoblox Host	infoblox.corp.example.com
Username	svc-ztf-infoblox
Password	<password or CyberArk reference>
DNS View	Internal
Network View	default

The connection to Foundation Central and Prism Central is not made directly by the browser UI. ZTF-Orchestrator writes or selects YAML config and then invokes the Nutanix ZeroTouch Framework. The ZeroTouch Framework reads the Foundation Central and Prism Central fields from the config/global settings and performs the API calls.

If Connection Profiles are configured, use the active profile to pre-fill Foundation Central, Prism Central, Prism Element, CVM, NCM, directory, DNS, NTP, and IPAM defaults in the guided workflow forms. Review the YAML Preview before saving or running the workflow.

8.3 Foundation Central Connection

Field	Example Value
Foundation Central IP	10.10.1.50
Foundation Central Username	admin
Foundation Central Password	<your FC admin password>

8.4 Prism Central Connection

Field	Example Value
Prism Central IP	10.10.1.10
Prism Central Username	admin
Prism Central Password	<your PC admin password>

8.5 Saving the Global Config

Click **Save and Preview** to see the rendered `global.yml` content before writing it to disk. Review it carefully to confirm all values are correct. Click **Save to File** to write the config to `<ZTF_PATH>/config/global.yml`.

9. Phase 7: Workflow Config Files

Each workflow execution requires its own YAML config file. ZTF-Orchestrator manages these files through the **Config Files** section of the sidebar.

9.1 Creating a Config File

1. Navigate to **Config Files**.
2. Click **New File**.
3. Enter a descriptive filename, for example **cluster-create-site1.yml**.
4. Paste or type the YAML content for your target workflow.
5. Click **Save**. The last five versions of each file are automatically backed up before any overwrite.

9.2 Config File Reference by Workflow

Cluster Create

```
# cluster-create-site1.yml
cluster_name: site1-cluster
cluster_vip: 10.10.1.21
node_ips:
  - 10.10.1.31
  - 10.10.1.32
  - 10.10.1.33
aos_version: "6.7.1"
ahv_version: "20230302.10004"
foundation_central_ip: 10.10.1.50
redundancy_factor: 2
```

Imaging Only

```
# imaging-site1.yml
foundation_central_ip: 10.10.1.50
aos_version: "6.7.1"
ahv_version: "20230302.10004"
nodes:
  - ipmi_ip: 10.10.1.101
    ipmi_user: ADMIN
    ipmi_password: ADMIN
    node_ip: 10.10.1.31
  - ipmi_ip: 10.10.1.102
    ipmi_user: ADMIN
    ipmi_password: ADMIN
    node_ip: 10.10.1.32
```

Configure Cluster

```
# configure-cluster-site1.yml
prism_element_ip: 10.10.1.21
cluster_name: site1-cluster
dns_servers:
  - 10.10.1.1
  - 10.10.1.2
ntp_servers:
  - ntp1.corp.example.com
  - ntp2.corp.example.com
smtp_server: smtp.corp.example.com
```


Deploy Prism Central

```
# deploy-pc-site1.yml
prism_element_ip: 10.10.1.21
pc_vm_name: PrismCentral
pc_ip: 10.10.1.10
pc_memory_gb: 30
pc_vcpus: 10
pc_storage_tb: 2
pc_version: "pc.2024.1"
network_name: VM-Network
```

Configure Prism Central

```
# configure-pc-site1.yml
prism_central_ip: 10.10.1.10
dns_servers:
  - 10.10.1.1
ntp_servers:
  - ntp1.corp.example.com
smtp_server: smtp.corp.example.com
admin_email: noc@corp.example.com
```

Site Deploy (End-to-End)

```
# site-deploy-site1.yml
site_name: site1
cluster_vip: 10.10.1.21
prism_central_ip: 10.10.1.10
foundation_central_ip: 10.10.1.50
aos_version: "6.7.1"
ahv_version: "20230302.10004"
node_ips:
  - 10.10.1.31
  - 10.10.1.32
  - 10.10.1.33
dns_servers:
  - 10.10.1.1
ntp_servers:
  - ntp1.corp.example.com
```

Full schema reference: See the ZeroTouch Framework documentation at <https://github.com/nutanixdev/zerotouch-framework> for the complete field list and optional parameters for every workflow type.

10. Phase 8: Running Workflows

Before running workflows, confirm the active Settings connection profile matches the target environment. Guided forms use that profile for defaults, but the generated YAML remains editable and should be reviewed before execution.

10.1 Single Workflow Execution

1. Navigate to **Workflows** in the sidebar.

2. Select the target workflow, for example **Cluster Create**.
 3. Select your config file from the dropdown.
 4. Click **Dry Run** first. This validates the config and checks connectivity without executing any changes. Review the output in the terminal pane. Fix any errors before proceeding.
 5. Once the dry run passes, click **Run Workflow**.
 6. A durable execution job is created and picked up by a background worker. The execution modal opens with a live terminal that streams the persisted job log in real time, so the subprocess lifecycle is no longer owned by the browser request.
 7. On completion, the job status, return code, output log, user, timestamp, and execution history entry are retained. Operators can review queued, running, successful, failed, cancelled, and interrupted jobs through the job-backed execution model.
1. If an execution must be stopped, use the cancel action where available. Cancellation is recorded as a job event and audit event.

10.2 Re-running a Previous Execution

1. Navigate to **Executions** in the sidebar.
2. Expand the execution record you want to repeat.
3. Click **Re-run**. The original config is loaded automatically, and the execution modal opens immediately, with no need to re-enter configuration.

10.3 Multi-Script Execution

Use the Script Library to queue and run multiple ZTF atomic scripts in sequence.

1. Navigate to **Scripts** in the sidebar.
2. Search or browse the 61 available ZTF atomic scripts across 12 categories.
3. Click each script to add it to the queue. Numbered badges show queue position in the list.
4. Reorder items using the up and down arrows.
5. Click **Run Queue**. All selected scripts are passed to ZTF as a single call (**--script A,B,C**).

Estimated phase progress is available for workflow and script jobs from v1.2.9. Treat the percentage as an operator-facing orchestration estimate based on local job state and observable ZTF output, not as a guaranteed Nutanix platform task percentage.

10.4 Pipeline Execution

Pipelines chain multiple workflows in sequence with automatic pass/fail gates.

1. Navigate to **Pipelines** in the sidebar.
2. Click **New Pipeline** and give it a name.
3. Add steps in the required order. Each step selects a workflow type and a config file.
4. Reorder steps using the up and down arrows.
5. Click **Run Pipeline**.
6. The pipeline modal displays a step progress rail showing status for each step: pending, running, success, failed, or skipped.

7. A failed step immediately halts the chain. All remaining steps are marked as skipped. This prevents partially successful deployments from continuing into subsequent dependent steps.

10.5 Scheduled Execution

Use Schedules to run recurring workflows with a standard 5-field cron expression. Each schedule stores its own workflow or script target, YAML config, enabled state, and last-run status.

Scheduled runs persisted in `schedules.json`, restored on restart, written to Execution History, and can trigger the configured webhook. Use Run Now to test a schedule before relying on the next cron interval.

10.6 Parallel Multi-Site Execution

Use Parallel Exec when the same workflow must run across several sites or clusters at the same time. A run accepts 2 to 10 site entries, each with a label and its own YAML configuration.

Review per-site terminal output and status after completion. Overall status is success when all sites pass, partial when some pass and some fail, and failed when all submitted sites fail.

10.7 Approval Gates

Use Approvals when operational policy requires admin review before a sensitive workflow is executed. Operators create requests with workflow, YAML, and notes; admins approve or reject with optional decision notes.

Requests expire automatically after 24 hours. Approval creation and decision events are retained and can notify the configured webhook endpoint.

10.8 Configuration Drift Detection

Use Drift Detection to compare a saved desired config with either the last successful applied configuration or a pasted observed-state JSON/YAML snapshot from Prism Central, Foundation Central, or another source.

Matched means no differences were found. Drifted means one or more fields are changed, missing, or unexpected. Unknown means no suitable last-applied baseline exists for the selected config/workflow.

11. Phase 9: Production Hardening

Packer/QCOW2 validation

If distributing a prebuilt AHV appliance, validate the Packer QCOW2 build, AHV image import, cloud-init or first-boot behavior, Docker service startup, PostgreSQL persistence, admin password retrieval, and rollback/backup process before publishing the image as a release artifact.

11.1 Automatic Startup with systemd (Linux)

Create a systemd service unit so ZTF-Orchestrator starts automatically on boot:

```
# /etc/systemd/system/ztf-orchestrator.service
[Unit]
Description=ZTF Orchestrator
After=network.target

[Service]
Type=simple
User=<youruser>
WorkingDirectory=/home/<youruser>/ztf/ZTF-Orchestrator
Environment="ZTF_PATH=/home/<youruser>/ztf/zerotouch-framework"
Environment="ZTF_PYTHON=/home/<youruser>/ztf/venv/bin/python3"
Environment="ZTF_PORT=5001"
ExecStart=/home/<youruser>/ztf/venv/bin/python3 server.py
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now ztf-orchestrator
sudo systemctl status ztf-orchestrator
```

The repository includes a `.dockerignore` file to keep Docker build contexts small and avoid OneDrive/cloud-file build-context errors when the repo is stored in a synced folder.

11.2 Automatic Startup with Docker

The `docker-compose.yml` already includes `restart: unless-stopped`. After any host reboot, the container will start automatically provided the Docker service itself is enabled:

```
sudo systemctl enable docker
```

11.3 TLS Reverse Proxy with nginx

The Flask server binds to `127.0.0.1` only and is not reachable over the network by default. For team access over the network, place nginx in front with TLS:

```
server {
    listen 443 ssl;
    server_name ztf.corp.example.com;

    ssl_certificate /etc/ssl/certs/ztf.crt;
    ssl_certificate_key /etc/ssl/private/ztf.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
```

```

location / {
    proxy_pass      http://127.0.0.1:5001;
    proxy_set_header    Host $host;
    proxy_set_header    X-Real-IP $remote_addr;
    proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_buffering      off;    # required for SSE live terminal output
    proxy_read_timeout   3700s;  # must exceed ZTF_EXEC_TIMEOUT (default 3600s)
    proxy_connect_timeout 10s;
}

server {
    listen 80;
    server_name ztf.corp.example.com;
    return 301 https://$host$request_uri;
}

```

Critical: `proxy_buffering off` is still recommended for responsive live terminal output. In v1.2.9, execution logs are persisted by the job worker, so output can also be reviewed after the job completes. This also helps live estimated progress updates reach the browser promptly.

After placing nginx in front, also update the Flask server to bind to **0.0.0.0** if nginx is on a different host. If nginx is on the same host (recommended), no change is needed -- nginx reaches Flask over localhost.

11.4 Firewall Rules (Linux ufw example)

```

# Allow only your team network to reach the nginx HTTPS port
sudo ufw allow from 10.10.0.0/16 to any port 443
sudo ufw deny 443
sudo ufw allow from 10.10.0.0/16 to any port 80
sudo ufw deny 80
# Block direct access to Flask (belt and braces, since it is already on 127.0.0.1)
sudo ufw deny 5001
sudo ufw enable

```

11.5 Log Rotation

ZTF-Orchestrator writes structured JSON logs to **<ZTF_DATA_DIR>/ztf-orchestrator.log**. To prevent unbounded growth, configure logrotate:

```

# /etc/logrotate.d/ztf-orchestrator
/home/<youruser>/.ztf-ui/ztf-orchestrator.log {
    daily
    rotate 30
    compress
    missingok
    notifempty
    copytruncate
}

```

Connection profiles are stored in settings.json under ZTF_DATA_DIR. Include settings.json in operational backups because it contains runtime paths, repository source, active profile selection, profile endpoint defaults, and webhook configuration.

12. Environment Variable Reference

Copy `.env.example` to `.env` in the ZTF-Orchestrator directory and adjust values. The server reads `.env` automatically on startup. Never commit `.env` to version control.

Variable	Default	Purpose
ZTF_DATA_DIR	~/.ztf-ui	File-backed state location for users, history, settings, configs, schedules, parallel runs, approvals, drift results, jobs, and logs
ZTF_PATH	~/zerotouch-framework	Path to the cloned ZTF directory
ZTF_PYTHON	current Python in PATH	Python executable used to invoke ZTF
ZTF_PORT	5001	Flask listen port
ZTF_EXEC_TIMEOUT	3600	Kill a workflow process after this many seconds
ZTF_TOKEN_TTL	28800	Session token lifetime in seconds (default is 8 hours)
ZTF_LOG_LEVEL	INFO	Logging verbosity: DEBUG, INFO, WARNING, or ERROR
ZTF_CONFIG_BACKUPS	5	Number of backup versions retained per config file
ZTF_MAX_BODY	1048576	Maximum request body size in bytes (default is 1 MB)
ZTF_EXEC_WORKERS	1	Number of background workers that process durable execution jobs
ZTF_STORAGE_BACKEND	postgres in Docker, file otherwise	State backend: postgres or file
ZTF_DATABASE_URL	postgresql://ztf:ztf@postgres:5432/ztf_orchestrator	PostgreSQL connection string when ZTF_STORAGE_BACKEND=postgres
POSTGRES_PASSWORD	ztf	Bundled Docker PostgreSQL password; change before production use
ZTF_AUDIT_RETENTION_DAYS	90	Retention window for structured audit events
ZTF_EXECUTION_RETENTION_DAYS	180	Retention window for execution history, drift, approvals, parallel runs, and jobs

Docker-specific variables (set as build args in `docker-compose.yml`):

Variable	Default	Purpose
ZTF_REPO_URL	GitHub URL	Git URL to clone ZTF from during image build
ZTF_REF	main	Git branch or tag to check out during image build

13. Air-Gapped Deployment

Both ZTF and ZTF-Orchestrator can operate with no internet access after installation.

13.1 Prepare Packages on an Internet-Connected Machine

```
# Clone both repositories
git clone https://github.com/VirtuArchitect/ZTF-Orchestrator.git
git clone https://github.com/nutanixdev/zerotouch-framework.git

# Download all pip packages into local wheel directories
pip download -r ZTF-Orchestrator/requirements.txt \
  -d ./airgap-wheels/ztf-ui

pip download -r zerotouch-framework/requirements/prod.txt \
  -d ./airgap-wheels/ztf
```

Transfer the **ZTF-Orchestrator/**, **zerotouch-framework/**, and **airgap-wheels/** directories to the air-gapped host via USB, SCP, or your internal file transfer mechanism.

13.2 Install on the Air-Gapped Host

```
# Create virtual environment
python3 -m venv ~/ztf/venv
source ~/ztf/venv/bin/activate

# Install from local wheels, no internet required
pip install --no-index --find-links ./airgap-wheels/ztf-ui \
  -r ZTF-Orchestrator/requirements.txt

pip install --no-index --find-links ./airgap-wheels/ztf \
  -r zerotouch-framework/requirements/prod.txt
```

13.3 Air-Gap Notes

- The **dist/** directory inside ZTF-Orchestrator is pre-built and committed to the repository. No Node.js or npm install is needed at runtime.
- The ZeroTouch Framework ships bundled Calm DSL wheels in the **calm-whl/** directory. No PyPI access is needed for Calm workflows.
- Set **ZTF_PATH** to point to your local copy of **zerotouch-framework**.

- For the in-app Setup page to work in an air-gapped environment, set the repository URL to your internal Git mirror URL and add that URL to **ALLOWED_REPOS** in **server.py**.
- For Docker air-gap, build the image on an internet-connected machine and export it:

```
docker build -t ztf-orchestrator:latest .
docker save ztf-orchestrator:latest | gzip > ztf-orchestrator.tar.gz
# Transfer the .tar.gz, then on the air-gapped host:
docker load < ztf-orchestrator.tar.gz
docker compose up -d
```

14. Troubleshooting

Symptom	Likely Cause	Resolution
"Python 3.10+ required" during install	Python version below 3.10	Install Python 3.10 or later and confirm it is on PATH
Dashboard shows ZTF path error (red)	ZTF_PATH not set or pointing to wrong directory	Set ZTF_PATH in environment or via Settings page
Workflow starts but fails immediately	ZTF_PYTHON points to wrong executable	Verify ZTF_PYTHON points to the venv Python, not the system Python
"Address already in use" on startup	Another process is using port 5001	Set ZTF_PORT=8080 in environment or .env file
Port 5001 fails on Windows	Port reserved by Hyper-V	Use ZTF_PORT=8080 instead
Live terminal is blank until job finishes	nginx buffering is enabled	Add proxy_buffering off to the nginx location block
Admin password not visible in terminal	Terminal scrolled or log redirected	Delete ~/ztf-ui/users.json and restart to regenerate
Foundation Central unreachable during workflow	Firewall blocking TCP 8000	Confirm TCP 8000 is open from the ZTF host to the FC VM
Docker: ZTF not found inside container	Stale image without ZTF baked in	Run docker compose build --no-cache && docker compose up -d
Config save fails with 403	Logged in as operator or viewer	Log in as admin to save or modify config files
Workflow returns exit code 1 immediately	Config file has invalid YAML	Navigate to Config Files, open the file, and fix YAML syntax errors
Session expires during a long workflow	ZTF_TOKEN_TTL too short	Increase ZTF_TOKEN_TTL in .env (default 28800 = 8 hours)
Audit Log shows no entries	Log level too high	Set ZTF_LOG_LEVEL=DEBUG in .env to increase verbosity
Schedule refuses to save	Invalid cron expression or unknown workflow/script	Use a standard 5-field cron expression and select only listed workflows or approved scripts
Scheduled run fails with missing config	Config file was moved, deleted, or outside the	Re-select or recreate the config file in the Config Files

	configured config directory	page before running the schedule again
Webhook not received for schedules or approvals	Webhook URL is blank or receiver rejects the POST	Confirm Settings > Notifications contains a reachable URL and review ztf-orchestrator.log for <code>webhook_failed</code>
Drift check shows Unknown	No successful baseline execution exists for the selected config/workflow	Run the workflow successfully first or use Snapshot mode with pasted current-state JSON/YAML
Docker build fails with invalid file request	Repository is in a OneDrive/cloud-synced path and Docker is packaging placeholder/reparse files	Ensure <code>.dockerignore</code> is present, keep dist assets pinned locally, or clone/build from a non-synced local path
Workflow form defaults are wrong	The wrong Settings connection profile is active	Open Settings > Connection Profiles, select the correct active profile, save settings, then reopen the workflow form

15. Field Readiness Checklist

Use this checklist at each deployment site before executing the first workflow.

Infrastructure

- ☐ Foundation Central VM deployed and powered on
- ☐ Foundation Central UI accessible at `http://<FC-IP>:8000`
- ☐ Foundation Central admin password changed from factory default
- ☐ Foundation Central registered with Prism Central (status: Registered)
- ☐ AOS package uploaded to Foundation Central File Server
- ☐ AHV package uploaded to Foundation Central File Server (AHV sites)
- ☐ Foundation package uploaded to Foundation Central File Server
- ☐ Imaging network has DHCP or static IPs documented for each node
- ☐ TCP 8000 open from ZTF host to Foundation Central
- ☐ TCP 9440 open from ZTF host to Prism Central
- ☐ TCP 9440 open from ZTF host to Prism Element (per cluster)

ZTF-Orchestrator Installation

☐ Appliance deployment method selected where appropriate (Docker bootstrap or prebuilt QCOW2)

☐ If using Packer, QCOW2 build and AHV import tested on a Linux build VM

☐ No real `.env`, PostgreSQL password, admin password, or Nutanix credentials included in the image or repository

- ☐ Python 3.10+ installed on ZTF host

- ☐ ZTF-Orchestrator installed (one-command, Docker, appliance, or manual)
- ☐ ZeroTouch Framework cloned and dependencies installed
- ☐ Server starts without errors on http://localhost:5001
- ☐ Admin password copied from first-run terminal output
- ☐ Personal admin account created
- ☐ Default admin account password changed or account deleted

Configuration

- ☐ Settings page: ZTF Path shows green
- ☐ Settings page: Python Executable shows green
- ☐ Settings > Connection Profiles: active profile matches the target environment
- ☐ Active profile contains Prism Central, Foundation Central, Prism Element/CVM, NCM, DNS, NTP, and IPAM defaults where required

- ☐ Settings page: Config Directory shows green
- ☐ Dashboard: all system health indicators green
- ☐ Global Config: vault type selected and saved
- ☐ Global Config: IPAM type selected and saved
- ☐ Global Config: Foundation Central IP and credentials saved
- ☐ Global Config: Prism Central IP and credentials saved
- ☐ global.yml written to <ZTF_PATH>/config/global.yml
- ☐ At least one workflow config file created in Config Files

Pre-Execution Validation

- ☐ Dry Run passes for the target workflow with zero errors
- ☐ Target nodes are powered on and IPMI is reachable (imaging workflows)
- ☐ Sufficient AOS/AHV packages available in Foundation Central (imaging workflows)
- ☐ IP addresses in config file are correct and not already assigned

Production Readiness (team deployments)

- ☐ systemd service or Docker restart configured for automatic startup
- ☐ nginx TLS reverse proxy configured with proxy_buffering off
- ☐ Firewall rules restrict access to trusted networks only
- ☐ Log rotation configured
- ☐ Schedules reviewed and disabled/enabled as intended
- ☐ Approval workflow owners identified for sensitive operations
- ☐ Drift detection baseline strategy agreed (last applied or snapshot)
- ☐ Webhook notification URL configured (optional)
- ☐ Operator and viewer accounts created for team members
- ☐ Backup scheduled for PostgreSQL volume/database and <ZTF_DATA_DIR> file assets (configs/, logs, and file-backed JSON state such as users.json, settings.json, schedules.json, parallel_runs.json, approvals.json, drift.json, and jobs.json where applicable)

ZTF-Orchestrator is developed and maintained by John Goulden.

ZeroTouch Framework is developed by Nutanix DevEx. Foundation Central is a Nutanix product.

v1.2.9 Update: NKP Binary and Readiness Preparation

ZTF-Orchestrator v1.2.9 adds operator-facing NKP deployment preparation controls. These controls do not replace Nutanix NKP installation prerequisites, but they make the Orchestrator a safer place to register NKP framework paths, prepare deployment profiles, and confirm binary readiness before approval-gated execution.

- Use the NKP Framework page to install or update the optional VirtuArchitect/nkp-zero-touch-framework integration before building NKP deployment profiles.
- Stage large NKP binaries or bundles on the Linux VM/appliance filesystem, then register the absolute path in the NKP Binary Manager. Smaller bundles may be uploaded through the UI when they fit within `ZTF_NKP_BINARY_MAX_UPLOAD`.
- Set the correct binary as the default, or apply a managed binary path directly to a specific NKP deployment profile.
- Check the Dashboard Deployment Readiness panel before execution. It now reports ZTF readiness, optional NKP framework readiness, generated NKP profiles/configs, and available NKP binaries.
- Use approval gates for controlled NKP phases. The Orchestrator records approval metadata, linked job IDs, task IDs when exposed by output, and audit events for traceability.

Operational note: for production-like AHV deployments, keep NKP binaries on persistent appliance storage or a mounted artifact share. Do not commit vendor binaries, secrets, or generated environment files to GitHub.